

Error and Exceptions

Introduction:

- Errors are problems in the program that the program should not recover from. If at any point in the program an error occurs, then the program should exit gracefully.
- On the other hand, Exceptions are raised when an external event occurs which in some way changes the normal flow of the program.
- When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

Handling Exceptions with Try/Except/Finally:

- Errors and Exceptions in Python are handled with the Try: Except: Finally construct.
- You put the unsafe code in the try: block.
- You put the fall-back code in the Except: block.
- The final code is kept in the Finally: block

Handling Exceptions with Try/Except/Finally:

- For example, look at the code below.

```
>>> try:
...     print("in the try block")
...     print(1/0)
... except:
...     print("In the except block")
... finally:
...     print("In the finally block")
...
in the try block
In the except block
In the finally block
```

Example: Try Block

- The try block will generate an exception, because x is not defined:

```
try:  
    print(x)  
except:  
    print("An exception occurred")
```

- Output:
An exception occurred

Example: Except Block

- Since the try block raises an error, the except block will be executed.
- Without the try block, the program will crash and raise an error:
- This statement will raise an error, because x is not defined:

```
print(x)
```

- Output:

```
Traceback (most recent call last):  
  File "demo_try_except_error.py", line 3, in <module>  
    print(x)  
NameError: name 'x' is not defined
```

Many Exceptions

- You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:
- Print one message if the try block raises a `NameError` and another for other errors:

```
try:  
    print(x)  
except NameError:  
    print("Variable x is not defined")  
except:  
    print("Something else went wrong")
```

```
Variable x is not defined
```

Else

- You can use the else keyword to define a block of code to be executed if no errors were raised:
- In this example, the try block does not generate any error:

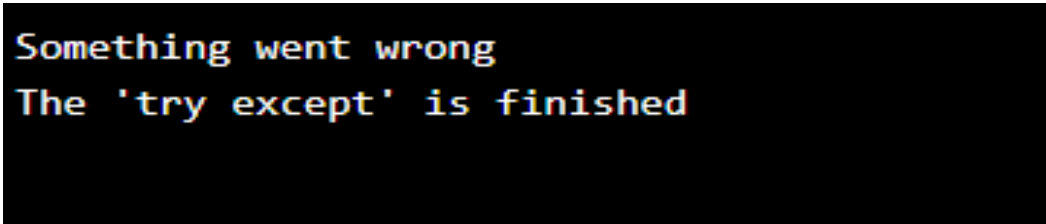
```
try:  
    print("Hello")  
except:  
    print("Something went wrong")  
else:  
    print("Nothing went wrong")
```

```
Hello  
Nothing went wrong
```

Example: Finally

- The finally block, if specified, will be executed regardless if the try block raises an error or not.

```
try:  
    print(x)  
except:  
    print("Something went wrong")  
finally:  
    print("The 'try except' is finished")
```



```
Something went wrong  
The 'try except' is finished
```

Raise an exception:

- As a Python developer you can choose to throw an exception if a condition occurs.
- To throw (or raise) an exception, use the raise keyword.
- The raise keyword is used to raise an exception.
- You can define what kind of error to raise, and the text to print to the user.

Output:

```
x = -1

if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

```
Traceback (most recent call last):
  File "demo_ref_keyword_raise.py", line 4, in <module>
    raise Exception("Sorry, no numbers below zero")
Exception: Sorry, no numbers below zero
```